ANÁLISE DE DADOS · UC 16682 · 2026/27

Mini-projeto · Dados em Streaming

Vigia da Câmara 7

Um dashboard web que vigia, em tempo real, uma câmara frigorífica de fruta e avisa quando alguma coisa corre mal

Grupos	2 a 3 alunos	**Lançamento**	29 de outubro de 2026
Entrega	12 de novembro de 2026, 23h59, no Moodle	**Defesa**	Aula prática seguinte à entrega, 8 min por grupo
Classificação	0 a 20 valores (+1 de bónus)	**Tecnologia**	Python no servidor, HTML/JS no browser

1. A história

A Cooperativa Frutícola da Cova da Beira guarda, entre junho e março, várias toneladas de cereja, pêssego e maçã em câmaras frigoríficas. A Câmara 7 é a mais antiga. Funciona, mas tem manias: o compressor faz ciclos irregulares, a porta fica aberta mais tempo do que devia quando o empilhador entra e, de vez em quando, o sensor de temperatura começa a dizer disparates.

No inverno passado, uma avaria no compressor aconteceu numa sexta à noite. A temperatura subiu devagar, meio grau por hora, e ninguém deu por nada até segunda de manhã. Perderam-se onze paletes de maçã. O problema não foi falta de dados: o sensor estava a enviar leituras de segundo a segundo. O problema é que ninguém estava a olhar para elas, e mesmo que estivesse, uma subida tão lenta não salta à vista num gráfico.

A direção da cooperativa pediu-vos uma coisa simples de dizer e menos simples de fazer: **um painel web que mostre o que se passa na câmara agora mesmo e que grite quando é preciso gritar**. Gritar demais também é um problema. Se o sistema dispara um alarme sempre que alguém abre a porta, ao fim de uma semana os operadores desligam o som e voltamos ao ponto de partida.

Como não vos vamos dar acesso à câmara verdadeira, vão começar por construir um **simulador do sensor** que se porte como ela, com as manias todas incluídas. Depois constroem o sistema que o vigia.

> **Porque é que isto é um problema de streaming**
>
> Não há um ficheiro CSV para abrir no fim. Os dados chegam um a um, para sempre, e cada decisão tem de ser tomada com o que já chegou. Não podem guardar tudo em memória, não podem voltar atrás para recalcular a média do dia e não podem esperar pelo fim do dia para decidir se há um problema. Tudo o que vimos na aula (estatísticas incrementais, janelas deslizantes, deteção de alterações) entra aqui.

2. O que se espera que aprendam

No final do mini-projeto deverão ser capazes de:

1. Gerar um fluxo de dados sintético realista, com ruído, sazonalidade curta, eventos e falhas, de forma reprodutível a partir de uma semente;

2. Processar um fluxo leitura a leitura, mantendo estatísticas incrementais (média, variância, mínimo, máximo, EWMA) sem guardar o histórico completo;
3. Usar janelas deslizantes de tamanho fixo e perceber o compromisso entre reagir depressa e reagir bem;
4. Desenhar regras de alerta que distingam um evento normal (a porta abriu) de uma situação perigosa (o compressor falhou), usando persistência, histerese e taxa de variação;
5. Detetar falhas do próprio sensor, que são diferentes de anomalias no processo físico;
6. Levar resultados de um servidor para o browser em tempo real (WebSocket ou Server-Sent Events) e apresentá-los de forma útil a quem toma decisões.

3. A visão geral

O sistema tem três peças. Cada uma corre como um processo independente, o que é importante: o servidor não pode espreitar o código do simulador para saber quando vem a avaria. Tem de a descobrir pelos dados, como faria na vida real.

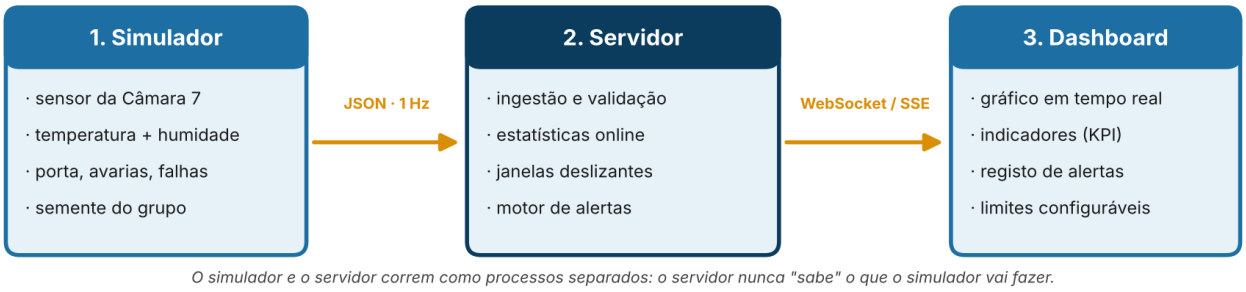


Figura 1. Arquitetura mínima do sistema.

Sugerimos **FastAPI** (ou Flask) no servidor, **WebSockets** ou **SSE** para empurrar dados para o browser e **Chart.js**, **Plotly.js** ou **ECharts** no dashboard. O simulador pode falar com o servidor por HTTP POST, por WebSocket ou por um broker MQTT (Mosquitto), à vossa escolha. Streamlit não é aceite como solução principal, porque esconde precisamente a parte que queremos que aprendam: o envio contínuo de dados do servidor para o cliente.

4. Peça 1: o simulador do sensor

O simulador envia uma leitura por segundo. Para não terem de esperar horas pela avaria, deve aceitar um fator de aceleração (por exemplo, velocidade 60 faz com que um minuto simulado passe num segundo real). Cada mensagem é um objeto JSON com este formato:

```
{
  "sensor_id":      "CAM07-T01",
  "seq":            18342,
  "ts":             "2026-11-05T14:03:27.512Z",
  "temperatura_c":  2.37,
  "humidade_pct":   91.4,
  "porta_aberta":   false
}
```

O campo `seq` é um contador que nunca deve repetir nem saltar. Se repetir ou saltar, o servidor fica a saber que alguma coisa se perdeu ou duplicou pelo caminho, e isso é informação que deve aproveitar.

4.1 O comportamento normal

Uma câmara saudável não tem uma temperatura constante. Ela oscila. O vosso simulador tem de reproduzir pelo menos estes efeitos, somados:

- **Setpoint.** A temperatura alvo da câmara, que depende do grupo (ver secção 4.3).
- **Ciclo do compressor.** O compressor liga e desliga, o que dá uma oscilação lenta de cerca de ±0,4 °C com período entre 12 e 20 minutos. Uma sinusoide serve, mas um dente de serra (sobe devagar, desce depressa) é mais fiel à realidade.
- **Ruído de medição.** Ruído gaussiano com desvio padrão σ, também definido pelo grupo.
- **Humidade.** Entre 88% e 95%, com variação lenta e inversamente relacionada com a temperatura.

4.2 As manias da Câmara 7

Aqui está a parte interessante. Os eventos seguintes têm de estar todos implementados e devem poder ser ligados ou desligados por parâmetro.

Evento	O que acontece aos dados	O que o sistema deve fazer
Porta aberta	porta_aberta passa a true durante 30 s a 3 min. A temperatura sobe depressa (até +3 °C) e volta ao normal em 5 a 10 minutos depois de fechar.	Não disparar alarme crítico. É normal. No máximo, um aviso se a porta ficar aberta mais de 2 min.
Avaria do compressor	A partir de um instante, a temperatura passa a subir de forma lenta e contínua, 0,3 a 0,6 °C por hora, por cima do comportamento normal.	Detetar antes de a temperatura ultrapassar o limite, se possível. É o evento que custou as onze paletes.
Valor preso	O sensor envia exatamente o mesmo valor durante muitos minutos.	Alerta de falha do sensor, não de temperatura.
Picos impossíveis	Leituras isoladas absurdas (−40 °C, 85 °C, ou null).	Rejeitar a leitura, contar, e alertar se forem frequentes. Nunca deixar estragar as estatísticas.
Silêncio	O simulador deixa de enviar durante 20 a 60 segundos e depois retoma.	Alerta de sensor sem comunicação. O dashboard deve mostrar claramente que os dados estão parados.
Duplicados e desordem	Cerca de 1% das mensagens chega repetida ou trocada com a seguinte.	Descartar duplicados pelo seq e lidar com a ordem.

4.3 A assinatura do grupo

Cada grupo tem a sua câmara. Calculem a **semente** somando os números de aluno dos elementos do grupo. Todos os parâmetros abaixo saem dessa semente, e o gerador aleatório do simulador deve ser inicializado com ela. Dois grupos diferentes não vão ter os mesmos dados, nem os mesmos limites, nem a avaria no mesmo minuto.

Parâmetro	Fórmula	Exemplo (semente 104 517)
Setpoint (°C)	1,0 + (semente mod 5) × 0,5	2,0
Limite superior (°C)	setpoint + 2,0	4,0
Limite inferior (°C)	setpoint − 2,0	0,0
Ruído σ (°C)	0,10 + (semente mod 4) × 0,05	0,15
Intervalo médio entre aberturas de porta (min)	8 + (semente mod 8)	13
Início da avaria (min simulados)	20 + (semente mod 25)	37
Falha de sensor na demonstração	semente mod 3 → 0: valor preso · 1: picos · 2: silêncio	0: valor preso

O simulador deve ter um modo `--cenario demo` que, a partir da semente, corre cerca de 90 minutos simulados e inclui pelo menos três aberturas de porta, a avaria do compressor no minuto indicado e a falha de sensor do vosso grupo. É este cenário que vão mostrar na defesa.

5. Peça 2: o servidor

O servidor recebe as leituras, decide se são válidas, atualiza o que sabe sobre a câmara e decide se é preciso alertar alguém. Tudo isto acontece a cada mensagem, e tem de acontecer depressa.

5.1 Ingestão e validação

- Validar o esquema da mensagem. Uma mensagem malformada é registada e descartada, nunca deita o servidor abaixo.
- Rejeitar leituras fisicamente impossíveis (temperatura fora de −30 °C a 50 °C, humidade fora de 0% a 100%). Estas leituras não entram nas estatísticas.
- Detetar duplicados e falhas na sequência através do seq. Mantenham contadores de mensagens recebidas, rejeitadas, duplicadas e perdidas.

5.2 Estatísticas em tempo real

Aqui é proibido guardar todas as leituras numa lista e calcular `np.mean()` a cada mensagem. A memória usada pelo servidor tem de ser limitada, independentemente de quanto tempo estiver a correr. Implementem:

- **Estatísticas globais incrementais** de temperatura: média e variância pelo algoritmo de Welford, mínimo e máximo.
- **Janela deslizante de 5 minutos** (em tempo simulado): média, desvio padrão, mínimo e máximo. Um `collections.deque` com tamanho máximo resolve a maior parte do problema.
- **EWMA** (média móvel exponencial) da temperatura, com α à vossa escolha. Expliquem a escolha no relatório.
- **Tendência**: inclinação da temperatura em °C/hora, estimada por regressão linear sobre a janela dos últimos 30 minutos. Este é o número que vai apanhar a avaria lenta.

5.3 O motor de alertas

Este é o coração do projeto e a parte que mais pesa na nota. Um alerta tem um nível, uma causa, um instante de início e um estado. Os níveis são três:

Nível	Significado
INFO	Algo aconteceu e ficou registado, mas ninguém precisa de se levantar. Exemplo: porta aberta.
AVISO	Atenção. Se continuar assim, vai ser um problema. Exemplo: tendência de subida anormal, porta aberta há mais de 2 minutos.
CRÍTICO	Alguém tem de agir agora. Exemplo: temperatura acima do limite de forma sustentada, sensor sem comunicação.

O motor deve implementar, no mínimo, estas regras:

- **R1 · Limite com persistência.** Crítico se a temperatura (ou a EWMA) estiver fora dos limites durante mais de N minutos seguidos. Um único valor fora dos limites não chega.
- **R2 · Histerese.** Um alerta de limite só fecha quando a temperatura voltar a estar pelo menos 0,5 °C dentro da zona segura. Sem isto, um valor a oscilar junto ao limite abre e fecha o alerta dezenas de vezes.
- **R3 · Tendência.** Aviso se a inclinação estimada se mantiver acima de um valor que escolham (por exemplo 0,25 °C/h) durante um período razoável, e a porta estiver fechada.
- **R4 · Porta.** Info quando a porta abre, aviso se ficar aberta mais de 2 minutos. Enquanto a porta está aberta e nos minutos seguintes, as regras R1 e R3 devem ser mais tolerantes.
- **R5 · Saúde do sensor.** Crítico se não chegar nenhuma mensagem há mais de 10 segundos reais; aviso se o desvio padrão das últimas 30 leituras for zero (valor preso); aviso se a taxa de leituras rejeitadas na janela passar de 5%.

Cada alerta passa por três estados: **ativo**, **reconhecido** (um operador carregou num botão a dizer que já viu) e **resolvido** (a condição desapareceu). Um alerta não pode ser disparado outra vez enquanto o anterior da mesma regra estiver ativo.

> **Uma boa pergunta para fazerem a vós próprios**
>
> Quanto tempo depois do início da avaria é que o vosso sistema dá o primeiro aviso? E quantos alarmes falsos deu durante a hora anterior? Estes dois números puxam em sentidos opostos. O relatório vai pedir-vos que os meçam.

6. Peça 3: o dashboard

Imaginem que o dashboard está num ecrã pendurado na parede do armazém e que quem o vê está a três metros de distância, com luvas e pressa. O que é que essa pessoa precisa de perceber em dois segundos? Desenhem a partir daí.

Requisitos obrigatórios:

- **Gráfico em tempo real** da temperatura nos últimos 15 minutos, que avança sozinho, com as linhas dos limites e a EWMA sobrepostas e os períodos de porta aberta assinalados (uma faixa sombreada resulta bem).
- **Indicadores principais**: temperatura atual, EWMA, tendência em °C/h, humidade, estado da porta, número de alertas ativos. O estado geral da câmara deve ser legível pela cor (verde, âmbar, vermelho) mas não só pela cor, porque há pessoas daltónicas.

- **Registo de alertas** com hora, nível, regra, mensagem e estado, com um botão para reconhecer.
- **Limites configuráveis**: o operador pode mudar os limites e o tempo de persistência a partir do dashboard. As alterações vão para o servidor e passam a valer para todos os clientes ligados.
- **Estado da ligação**: se o servidor cair ou o sensor ficar em silêncio, o ecrã tem de o mostrar de forma inequívoca. Um gráfico parado que parece normal é o pior cenário possível.
- **Memória limitada no browser**: o gráfico guarda um número fixo de pontos e descarta os mais antigos. Deixem o dashboard aberto uma hora e confirmem que o separador não fica cada vez mais lento.

Opcional, mas fica bem: um alerta sonoro para eventos críticos (que o operador possa silenciar), e uma vista com estatísticas da sessão (mensagens recebidas, rejeitadas, perdidas, latência média).

7. O que entregam

Um único ficheiro .zip no Moodle, com o nome AD-MP-Streaming-<semente>.zip, contendo:

- **Código** das três peças, organizado em pastas, com um requirements.txt.
- **README.md** que permita a qualquer colega pôr tudo a correr em menos de cinco minutos. Os nomes e números dos elementos do grupo e a semente ficam no topo.
- **Relatório** em PDF, com no máximo 4 páginas, que responda às perguntas da secção 8.
- **Um gráfico** (PNG) do cenário de demonstração completo, com a temperatura, os limites e os instantes de cada alerta marcados.

8. Perguntas para o relatório

O relatório não é para descrever o código. É para mostrar que perceberam o que construíram. Respondam a estas perguntas com números tirados do vosso próprio sistema:

Q1. No cenário de demonstração, quantos minutos passaram entre o início real da avaria e o primeiro alerta? Qual das regras disparou primeiro?

Q2. Quantos alertas foram disparados durante a primeira hora, quando não havia avaria nenhuma? Cada um deles era justificado?

Q3. Que valor de α escolheram para a EWMA e que valor de N para a persistência? Mostrem o que acontecia com um valor claramente mais baixo e um claramente mais alto.

Q4. Quanta memória usa o servidor ao fim de 10 minutos e ao fim de 2 horas de execução? Se os valores forem diferentes, porquê?

Q5. Qual é a latência média entre o instante em que o simulador gera a leitura e o instante em que ela aparece no browser? Como a mediram?

Q6. Se a cooperativa tivesse 200 câmaras em vez de uma, o que é que deixava de funcionar no vosso desenho e o que mudariam primeiro?

9. Como vai ser avaliado

Componente	Cotação	Peso
A. Simulador: comportamento normal, todos os eventos, assinatura do grupo, modo demo, aceleração	3,0	15%

Componente	Cotação	Peso
B. Ingestão e estatísticas: validação, seq, Welford, janela deslizante, EWMA, tendência, memória limitada	4,0	20%
C. Motor de alertas: regras R1 a R5, histerese, estados, ausência de alarmes repetidos	5,0	25%
D. Dashboard: tempo real, legibilidade, limites configuráveis, estado da ligação	4,0	20%
E. Relatório: respostas fundamentadas com números do vosso sistema	2,0	10%
F. Defesa: demonstração ao vivo do cenário demo e perguntas individuais	2,0	10%
Total	**20,0**	**100%**

Bónus (até +1 valor, sem ultrapassar os 20). Uma das seguintes: deteção da avaria com ADWIN ou Page-Hinkley a correr em paralelo com a regra R3, com comparação dos tempos de deteção; persistência do histórico em SQLite com consulta de períodos passados no dashboard; suporte a várias câmaras em simultâneo, cada uma com a sua semente.

Na defesa, cada elemento do grupo responde a pelo menos uma pergunta sobre uma parte do código que não foi a que escreveu. **A nota da defesa é individual** e pode ajustar a nota final de cada aluno em relação à do grupo.

10. Regras do jogo

- Podem usar ferramentas de IA generativa para vos ajudar, desde que o declarem no README (que ferramenta e para quê). O que não podem é entregar código que não conseguem explicar. Na defesa isso nota-se depressa.
- Conversar com outros grupos sobre ideias é saudável. Partilhar código não é. Como cada câmara tem a sua semente, dois relatórios com os mesmos números são fáceis de encontrar.
- Entregas atrasadas perdem 1 valor por cada dia, até um máximo de três dias. Depois disso o trabalho não é aceite.
- Se o vosso sistema não funcionar na defesa por um problema técnico de última hora, mostrem o gráfico entregue e expliquem o que deveria acontecer. Vale mais do que desistir.

11. Dicas de quem já se enganou

- **Comecem pelo fim.** Ponham um número qualquer a aparecer no browser, vindo do servidor, a cada segundo. Só depois disso funcionar é que vale a pena fazer o simulador bonito.
- **Separem o tempo simulado do tempo real.** As janelas e as regras de persistência funcionam sobre o campo ts. O detetor de silêncio funciona sobre o relógio do servidor. Misturar os dois é a fonte de erro mais comum neste tipo de trabalho.
- **Testem cada regra sozinha.** Liguem só a avaria e confirmem que a R3 dispara. Liguem só as portas e confirmem que a R1 não dispara. Juntar tudo logo à primeira dá um sistema que ninguém sabe porque se comporta como se comporta.

- **Não confiem no gráfico.** Uma subida de 0,4 °C por hora é invisível a olho num gráfico de 15 minutos. Por isso é que a tendência existe.
- **Um alarme que toca sempre é um alarme que ninguém ouve.** Se o vosso sistema dispara vinte alertas na primeira hora sem avaria nenhuma, não está pronto, por muito bonito que seja o dashboard.

> *Bom trabalho. E lembrem-se das onze paletes de maçã.*